

AI Code Refactoring: Letting a Probabilistic System Edit Production Code Safely

Understand the loop, the verification steps and the deterministic tooling that make autonomous refactoring trustworthy.

For engineers evaluating or building autonomous refactoring into their workflow · 26 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

[Watch the source video](#)[Read the full lesson](#)[Read the condensed version](#)

What matters

- ✓ Refactoring changes internal structure while leaving external behaviour identical — the second half is what makes it verifiable.
- ✓ The three canonical motives are readability, removing duplication and lowering complexity; skipping them compounds into technical debt.
- ✓ Inline refactoring works alongside a developer on small local edits; agentic refactoring is handed a goal and works the whole codebase autonomously.
- ✓ The danger is a probabilistic system deleting things that look unused but exist for a reason nobody wrote down.
- ✓ The guard rail is a loop: plan, read, search, report, approve, snapshot, patch, verify — with rollback on failure.
- ✓ Human approval sits between the report and any edit, so guesses never reach the codebase unreviewed.
- ✓ Because verification is just tests and builds, the whole loop slots into CI/CD like another developer.
- ✓ Patching via a syntax tree makes transformations like rename deterministic rather than probabilistic.

What it covers

- 01 Refactoring: change the inside, preserve the outside** 0:00
Take a program's internal structure and change it, while to the outside world the thing still looks exactly the same.
- 02 Why bother, and what happens if you do not** 0:43
Readability, removing duplication, lowering complexity. Skip them long enough and it compounds into technical debt.
- 03 Inline versus agentic refactoring** 2:50
One works next to you on small local edits. The other is handed a goal and works the whole codebase on its own.
- 04 The risk: a probabilistic system editing hundreds of files** 4:55
It predicts likely-looking code, so it may tidy away something that looked unused but was there for a reason.
- 05 The loop, part one: plan, read, search, report** 5:37
Agentic refactoring manages risk by working as a cycle. The first four stages build context and produce findings rather than edits.
- 06 The loop, part two: approve, snapshot, patch** 7:44
The human re-enters here, choosing which findings to fix. Then a snapshot is taken so everything can be undone, and only then does code change.
- 07 Verify, roll back, and the CI/CD fit** 8:27

The agent writes and runs tests and rebuilds the project. If a test breaks it rolls back and goes round again — which is why the whole loop fits into a pipeline.

08 **Deterministic patching with syntax trees** 9:53

The patch step can skip probabilistic generation entirely by parsing the code into a tree and operating on its structure.

09 **Learning from accept, reject, pass and fail** 10:35

Every approval, rejection, passing test and failing test is a training signal, so the suggestions improve over time.

Glossary

Refactoring — Changing a program's internal structure while leaving its external behaviour identical.

Technical debt — The compounding cost of deferred cleanup, paid as interest by every future change that has to route around the mess.

Inline refactoring — Small, local suggestions made in the editor alongside the developer, reviewed continuously as they appear.

The refactoring loop — Plan, read, search, report, approve, snapshot, patch, verify — with rollback on failure and another pass round.

Abstract syntax tree (AST) — A structural representation of parsed code. Discards formatting and comments, so regenerating source reformats the file.

Deterministic patching — Applying a change as a tree operation rather than generating text, making transformations like rename correct by construction.

Training signal — An outcome usable as a label. Here: whether a finding was approved, and whether the resulting patch passed the tests.

External behaviour — Everything observable from outside: return values, types, side effects, exceptions, emitted logs and timing. What a refactoring must preserve.

Characterisation test — A test asserting that behaviour has not changed rather than that it is correct. The right instrument before refactoring untested code.

Agentic refactoring — An autonomous agent given a goal that plans and executes changes across the whole codebase, reviewed after the fact.

Snapshot — The recorded starting state, in practice a branch or commit, taken automatically so any change can be undone.

Lossless syntax tree — A richer tree that preserves formatting, comments and whitespace exactly, so a transformation produces a minimal diff.

Dead code — Code with no reachable callers. Hard to establish statically, because reflection, schedulers, flags and config can reach it invisibly.

Explanations are AI-generated. Check anything you intend to rely on.